

The functions in lines 119 to 122 and 129 to 132 just initialise the mutex attribute and then apply those attributes to the shared mutex variable. The attributes can be protocol (how a thread behaves in terms of priority when a higher-priority thread wants the mutex) and robustness (what happens when you acquire a mutex and the original owner died while possessing it). `pthread_mutexattr_init` takes the address of `pthread_mutexattr_t` type variable as its argument. `pthread_mutex_init` takes the address of `pthread_mutex_t` type variable and `pthread_mutexattr_t` type variable as its argument.

The code on lines 123 to 126 is used to set the inheritance protocol for the mutex attribute variable. First the mutex attribute is initialised and then the protocol is set. The protocol value `PTHREAD_PRIO_INHERIT` makes a thread inherit the priority of a thread it is blocking.

The code in lines 134 to 140 is for thread creation. `pthread_create` is used to create threads in the calling process. `pthread_t` is the data type used to create thread variables. It takes four parameters as its argument. The first argument is the thread address, while the second argument is used to set the thread attributes such as stack size, scheduling policy and priority. If `NULL` is specified, then the default attributes are applied. The third argument is the function that the thread will execute and, finally, the fourth argument is the argument for the thread function. A single argument can be directly passed here, but if there is more than one argument, then we must declare a structure for arguments and pass the address of the structure.

`pthread_join` binds the threads to the main process so that it waits for the threads to finish execution. It is used to synchronise thread activities that work like a wait system call in process.

The functions `pthread_mutexattr_destroy` and `pthread_mutex_destroy` in lines 137 to 142 destroy a mutex and mutex attribute object; the object becomes uninitialised.

The name of the second file in this source code is `pri-inv-singlecore.c`. The

```
sourabh@sourabh-Lenovo-Y50-70: ~
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
H is running with priority 99 on cpu: 1
```

Figure 4: H is running

```
sourabh@sourabh-Lenovo-Y50-70: ~/PE/PE
sourabh-Lenovo-Y50-70# ./a.out

L is running with priority 1
M is running with priority 89
```

Figure 5: L is pre-empted by M

```
sourabh@sourabh-Lenovo-Y50-70: ~
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
H is running with priority 99
```

Figure 6: L executes again and then H runs

```
sourabh@sourabh-Lenovo-Y50-70: ~
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
L is running with priority 1
H is running with priority 99
M is running with priority 89
sourabh-Lenovo-Y50-70# █
```

Figure 7: Priority inheritance

code is the same as that for `pri-inh-singlecore.c` except that it doesn't have the priority inheritance protocol applied to `pthread_mutex`.

The third file in this source code is named `no-inv-multicore.c`. The code is the same as that for `pri-inh-singlecore.c` except for the changes mentioned above (`pri-inv-singlecore.c`), and also all the threads are made to run on different cores (1, 2 and 3).

## Results

**Setup:** The experiments were conducted for single-core and multi-core systems. The program consisted of three threads with different priorities. Let us denote them as L, M and H where L, M and H are lower, medium and higher priority threads respectively.

The real-time scheduling policy `SCHED_FIFO`, which is a first in, first out scheduling algorithm without time slices, is used. L and H share a critical section while M doesn't share anything with either of the threads.

**Execution on multi-core systems:** The execution of the program in a multi-core environment showed normal behaviour. Since more than one core is available for threads to run, M doesn't pre-empt L; instead, there is a simultaneous execution of L and M. When L finishes execution, it releases the mutex lock and, finally, H gets to execute.

**Execution in a single-core environment:** For single-core execution, the CPU affinity is set. CPU affinity binds a thread to a core. This forces all the threads to run on a single core.

The behaviour of the program is as follows. L starts execution; meanwhile, M and H arrive. H waits for the mutex lock and M starts executing by pre-empting L. When M finishes off, L continues and runs to completion. When the lock is released, H gets to execute and runs to completion. This happens because they are all running on a single core. M has a higher priority than L; so it can pre-empt L without waiting for any resources. H must wait because it shares the critical section with L, and can run only when L releases the mutex lock. This is what we call priority inversion. When H was supposed to run, in its turn, M gets to run.

**Using priority inheritance:** To overcome the above problem, the priority inheritance protocol